CO5 ASSESSMENT TOOL -2

NAME : SRAVANTHI K

REG NO : 192372127

COURSE CODE: CSA 4301

COURSE NAME : INTERNET PROGRAMMING

1. A company needs to develop a web service for a mobile and web application. Analyze both REST and SOAP approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

Answer:

REST and SOAP are two commonly used approaches for developing web services. Both can enable communication between mobile applications, web applications, and backend systems, but they differ in architecture, message format, performance, scalability, and security features.

REST (Representational State Transfer) is an architectural style that generally uses HTTP methods such as GET, POST, PUT, PATCH, and DELETE. REST commonly exchanges data using lightweight JSON, although other formats can also be used. It is stateless, meaning each request contains the information required to process it.

SOAP (Simple Object Access Protocol) is a protocol that uses XML-based messages. SOAP defines a structured message format and supports standards such as WS-Security, reliable messaging, and transaction-related specifications. It is suitable for systems that require strict contracts and advanced enterprise-level security and reliability.

Comparison:

1. Performance:
REST is generally faster because JSON messages are lightweight and REST uses standard HTTP operations. SOAP messages use XML, which is more verbose and requires additional processing. For mobile and web applications where low latency and reduced

data usage are important, REST is usually more suitable.

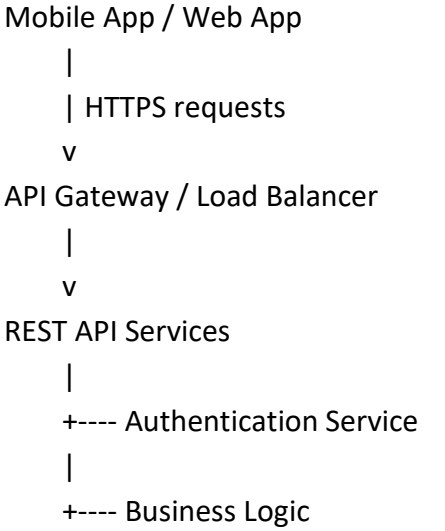
2. Scalability:
REST is stateless, which makes it easier to scale horizontally by adding multiple application servers behind a load balancer. Because the server does not need to maintain client session information between requests, requests can be distributed across servers efficiently. SOAP can also be scaled, but its additional processing and enterprise features may introduce more complexity.

3. Security:
SOAP provides strong built-in enterprise standards such as WS-Security, XML encryption, and XML signatures. REST normally relies on HTTPS/TLS together with authentication mechanisms such as OAuth 2.0, OpenID Connect, API keys, and JSON Web Tokens (JWT). For a mobile and web application, HTTPS combined with token-based authentication can provide strong practical security.

4. Flexibility:
REST supports multiple clients and is well suited to mobile applications, browsers, and JavaScript-based applications. JSON is also easy to process in most modern programming languages. SOAP is more rigid because it commonly relies on XML and formal service contracts.

Selected solution: REST API.

Design:



|
v
Database / Cache

The clients communicate with the REST API through HTTPS. An API gateway can handle authentication, request routing, rate limiting, logging, and traffic control. Multiple REST service instances can run behind a load balancer. The services communicate with the database and optionally use a cache such as Redis for frequently accessed information.

Security can be implemented using HTTPS/TLS, OAuth 2.0 or OpenID Connect, short-lived access tokens, input validation, rate limiting, and proper authorization checks.

Conclusion:
REST is the appropriate choice for this mobile and web application because it provides good performance, easy horizontal scalability, simple integration, and efficient data transfer. SOAP would be preferable if the application required strict enterprise contracts, WS-Security, complex transactions, or legacy SOAP-based integration.

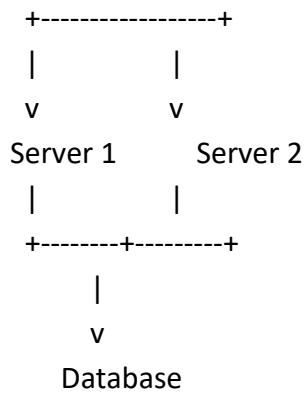
2. An online platform expects a large number of users accessing its services simultaneously. Design a scalable service architecture showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

Answer:

When a large number of users access an online platform at the same time, a single server and database can become a bottleneck. A scalable architecture should distribute requests across multiple servers and reduce unnecessary database load.

Basic scalable architecture:

Clients
|
v
Load Balancer
|
v
API Gateway
|

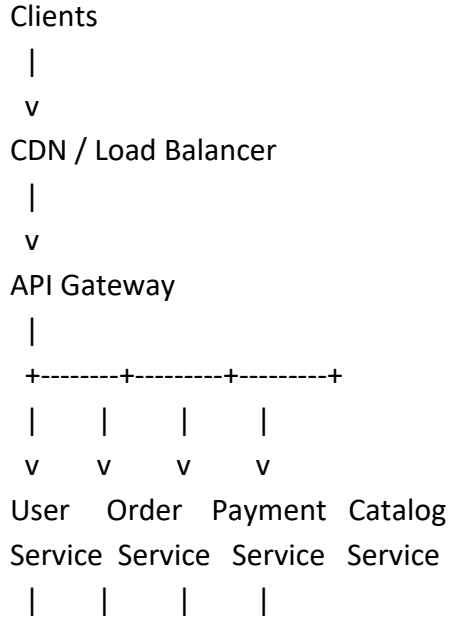


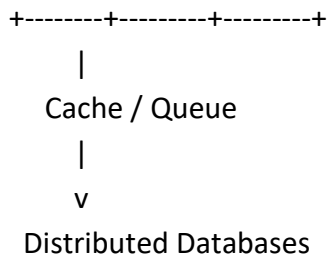
The client may be a web browser or mobile application. Requests are first received by a load balancer. The load balancer distributes traffic among multiple application servers. The API layer provides endpoints for operations such as user authentication, product retrieval, orders, and payments.

The application servers implement business logic and communicate with the database. The database stores persistent information such as user records and transaction data.

To improve scalability, the architecture can use horizontal scaling. Instead of continuously increasing the capacity of one server, multiple application server instances are added. A load balancer distributes incoming traffic among these instances.

Alternative design: Microservices with caching and asynchronous processing.





In this alternative design, the application is divided into independent services. Each service can be scaled according to its workload. For example, if the catalog receives much more traffic than the payment service, additional catalog instances can be deployed without scaling every other service.

A distributed cache can store frequently requested data and reduce database queries. A message queue can handle long-running operations asynchronously. A CDN can serve static files such as images, JavaScript, and CSS without sending every request to the application servers.

Database scalability can be improved using read replicas, database partitioning, indexing, and appropriate connection pooling.

Advantages:
1. Horizontal scaling allows additional servers to be added as demand increases.
2. Load balancing prevents one server from receiving all requests.
3. Caching reduces repeated database operations.
4. Microservices allow individual components to scale independently.
5. Message queues prevent temporary traffic spikes from overwhelming backend services.
6. CDN usage reduces load on application servers.

Conclusion:
The alternative microservices architecture with an API gateway, load balancing, caching, asynchronous queues, and scalable databases provides better scalability for a high-traffic online platform. It can handle traffic growth more efficiently than a single-server architecture.

3. A banking application requires secure communication between client and server. Analyze available communication protocols and select the most suitable protocol. Justify your selection based on security, reliability, and performance.

Answer:

A banking application handles highly sensitive information such as account details, authentication credentials, transaction information, and financial data. Therefore, communication between the client and server must provide confidentiality, integrity, authentication, and reliability.

Important communication choices include HTTP, HTTPS, TLS, TCP, and protocols such as WebSocket over TLS.

1. HTTP:
HTTP transfers data without encryption by itself. Sensitive banking information sent using plain HTTP can be intercepted or modified by attackers. Therefore, HTTP alone is unsuitable for banking communication.

2. HTTPS:
HTTPS is HTTP protected by TLS. It encrypts data during transmission and helps prevent eavesdropping and tampering. It also provides server authentication using digital certificates.

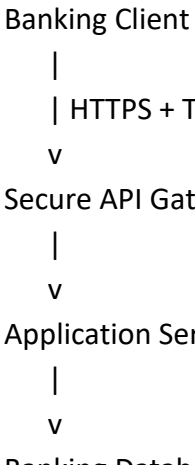
3. TLS:
TLS provides the security layer used by HTTPS. It provides encryption, integrity protection, and authentication. Modern deployments should use current secure TLS versions and disable obsolete protocols and weak cryptographic algorithms.

4. TCP:
TCP provides reliable, ordered delivery of network data, but TCP itself does not encrypt application data. It therefore needs to be combined with TLS when confidentiality and integrity are required.

5. WebSocket over TLS:
Secure WebSockets can be useful when the banking application requires persistent, real-time communication, such as live transaction notifications. However, it is not necessary for ordinary request-response banking operations.

Selected protocol: HTTPS using TLS.

Architecture:



Security mechanisms should include:
- HTTPS/TLS for encrypted communication.
- Strong server certificates and certificate validation.
- Multi-factor authentication for users.
- OAuth 2.0/OpenID Connect or another suitable secure authentication mechanism.
- Short-lived access tokens and secure session management.
- Authorization checks for every sensitive operation.
- Input validation and protection against common web attacks.
- Rate limiting and monitoring for suspicious activity.
- Secure logging without storing passwords or unnecessary sensitive information.

Reliability is improved because TCP, underneath HTTPS, provides ordered and reliable delivery. Application-level timeouts, retries where safe, transaction controls, and idempotency mechanisms should also be implemented.

Performance:
TLS adds some processing overhead, but modern TLS implementations are efficient. Persistent connections and connection reuse can reduce repeated connection costs. For banking systems, the small overhead is justified by the security benefits.

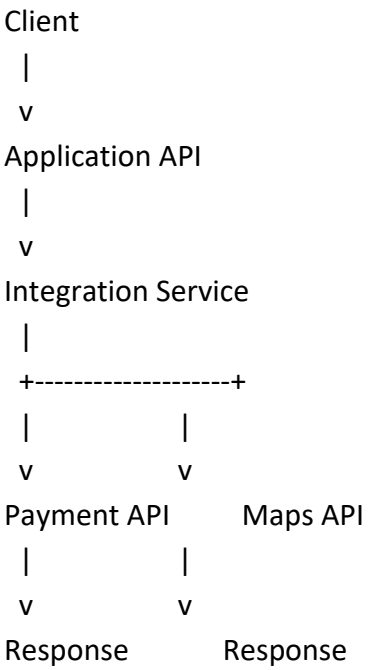
Conclusion:
HTTPS over modern TLS is the most suitable choice for normal banking client-server communication because it provides confidentiality, integrity, authentication, and reliable transport while maintaining good performance. Secure WebSockets over TLS can be added when real-time communication is required.

4. An application needs to integrate with third-party services such as payment gateways or maps. Design an API integration strategy, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.

Answer:

Third-party API integration allows an application to use external services such as payment gateways, maps, address verification, messaging, and other specialized services. A well-designed integration strategy should protect credentials, validate requests, handle failures, and process responses safely.

Proposed API integration architecture:



Request handling:
1. The client sends a request to the application's backend.
2. The backend authenticates and authorizes the user.
3. The application validates all input data.
4. The integration service constructs the third-party API request.
5. Required headers, parameters, and authentication credentials are added.
6. The request is sent over HTTPS.
7. The third-party service returns a response.
8. The integration service validates and transforms the response.

9. The application returns a safe and appropriate response to the client.

Authentication:
API keys, OAuth 2.0 access tokens, signed requests, or other mechanisms should be used according to the third-party provider's requirements. Secrets should not be embedded in mobile applications or frontend JavaScript. They should be stored securely on the backend, preferably using a secrets manager.

Request handling should also include:
- Input validation.
- Timeout configuration.
- Rate-limit handling.
- Request IDs for tracing.
- Idempotency keys for operations such as payments.
- Secure logging.
- Retry policies for temporary failures.

Response processing:
The application should check the HTTP status code and response body before treating a request as successful. External responses should be validated against expected schemas. Sensitive information should not be unnecessarily returned to the client.

Alternative approach: Asynchronous integration using a message queue.

For operations that do not need an immediate response, the application can place a task into a message queue. A worker then communicates with the third-party service.

Client
|
v
Application API
|
v
Message Queue
|
v
Integration Worker
|

v
Third-Party API

This approach improves reliability because temporary third-party failures do not necessarily block the main application request. Failed tasks can be retried, and dead-letter queues can be used for messages that repeatedly fail.

For payment operations requiring an immediate result, synchronous processing may still be necessary, but idempotency and carefully designed retry handling should be used.

Conclusion:
A centralized integration service provides consistent authentication, validation, monitoring, and error handling. For non-immediate operations, asynchronous queues and worker services can further improve reliability, scalability, and resilience.

5. A web service frequently encounters errors during client requests. Design an error handling and fault tolerance mechanism. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Answer:

A web service can experience errors because of invalid client input, network failures, unavailable dependencies, database problems, server overload, or unexpected application exceptions. A robust system should detect errors, handle them consistently, recover when possible, and provide meaningful responses without exposing sensitive internal information.

Proposed error handling architecture:

Client
|
v
API Gateway
|
v
Application Service
|
+------> Database
|

+------> External Service

Error handling mechanism:

1. Input validation:
The service should validate request parameters, data types, required fields, and business rules before processing the request. Invalid requests should return appropriate 4xx responses.

2. Standard error responses:
The API should use consistent error formats containing information such as an error code, a safe human-readable message, and a request or correlation ID.

3. HTTP status codes:
- 400 Bad Request for invalid input.
- 401 Unauthorized when authentication is required or invalid.
- 403 Forbidden when the user lacks permission.
- 404 Not Found when the requested resource does not exist.
- 409 Conflict for conflicting operations.
- 429 Too Many Requests when rate limits are exceeded.
- 500 Internal Server Error for unexpected server failures.
- 502/503/504 when appropriate for upstream or temporary service failures.

4. Logging and monitoring:
Detailed technical information should be recorded in secure server-side logs. Monitoring systems can track error rates, response times, availability, and dependency failures. Sensitive data such as passwords should never be logged.

5. Timeouts:
Requests to databases and external services should have defined timeouts so that a failed dependency does not block the server indefinitely.

6. Retry mechanism:
Temporary failures can be retried using controlled retries with exponential backoff and jitter. Retries should not be used blindly for non-idempotent operations because they may cause duplicate actions.

7. Circuit breaker:

A circuit breaker can temporarily stop requests to an unhealthy external dependency. This prevents repeated failures from consuming application resources and allows the dependency time to recover.

8. Graceful degradation:
If a non-critical service fails, the application should continue providing essential functionality. For example, a recommendation service can be unavailable while the main product service continues working.

Alternative strategies:

A. Redundancy and load balancing:
Run multiple application instances and use a load balancer. If one instance fails, traffic can be redirected to healthy instances.

B. Caching:
Frequently accessed data can be cached to reduce database load and improve response time.

C. Message queues:
Background operations can be placed in queues so temporary service failures can be retried without losing tasks.

D. Health checks:
Application instances should expose health information so load balancers and orchestration platforms can detect unhealthy instances.

E. Database replication and backups:
Database replicas can improve availability and backups can support disaster recovery.

F. Rate limiting:
Rate limits protect the service from excessive traffic and accidental or malicious request floods.

Smooth user experience:
Users should receive clear messages such as "The service is temporarily unavailable. Please try again." rather than technical stack traces. The interface can provide retry options for temporary failures and display transaction status when operations are

processed asynchronously.

Conclusion:
A combination of validation, standardized error responses, logging, monitoring, timeouts, controlled retries, circuit breakers, redundancy, caching, and graceful degradation creates a fault-tolerant web service. These mechanisms reduce downtime, prevent cascading failures, and provide users with a more reliable experience.